

三维云 CAD 产品

CrownCAD 二次开发使用讲解

www.crowncad.com



目录

CONTENTS

01 模块简介

02 语法规则

03 API 使用

04 示例程序

05 问题答疑

目录

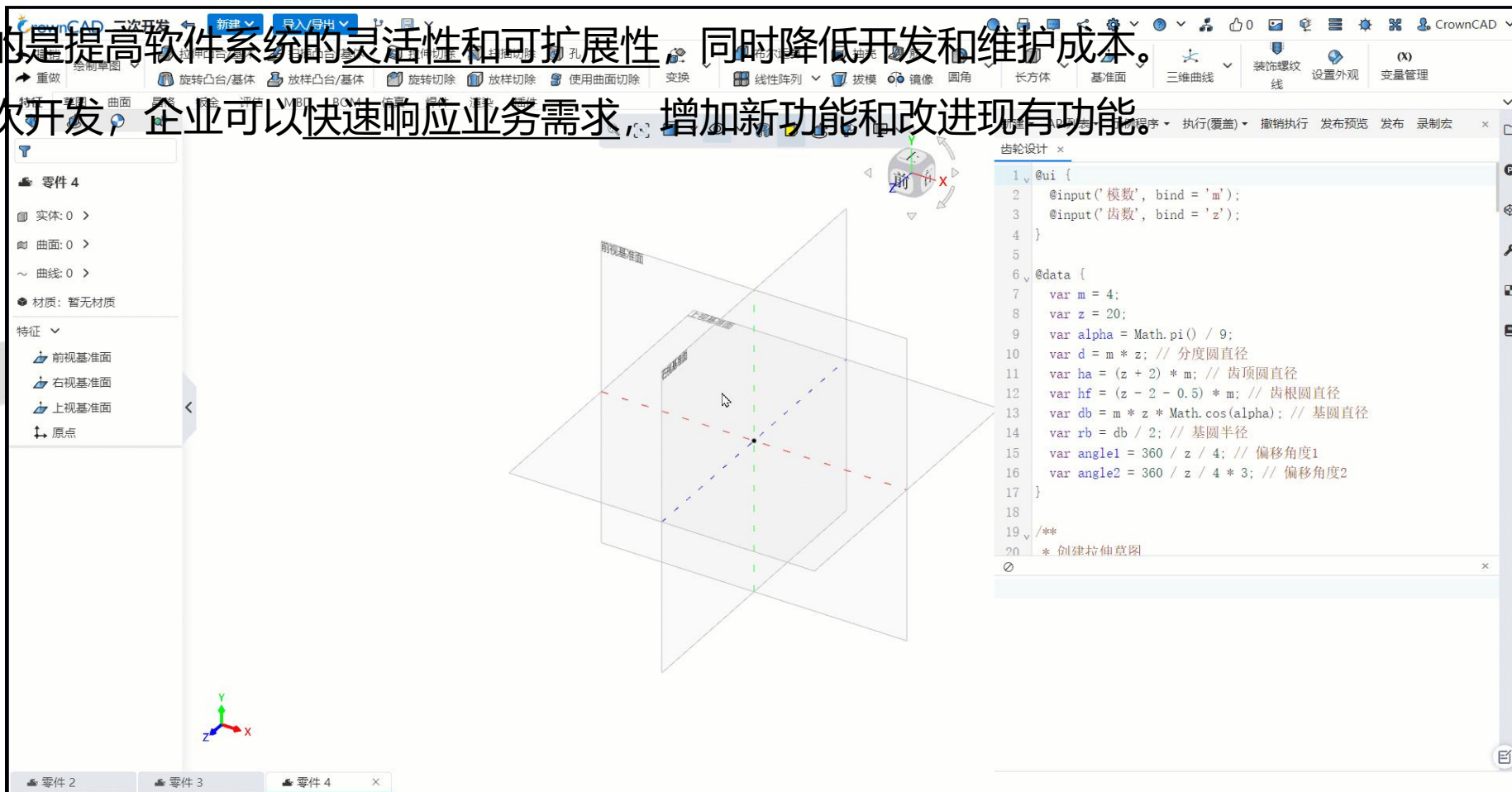
CONTENTS

01 模块简介



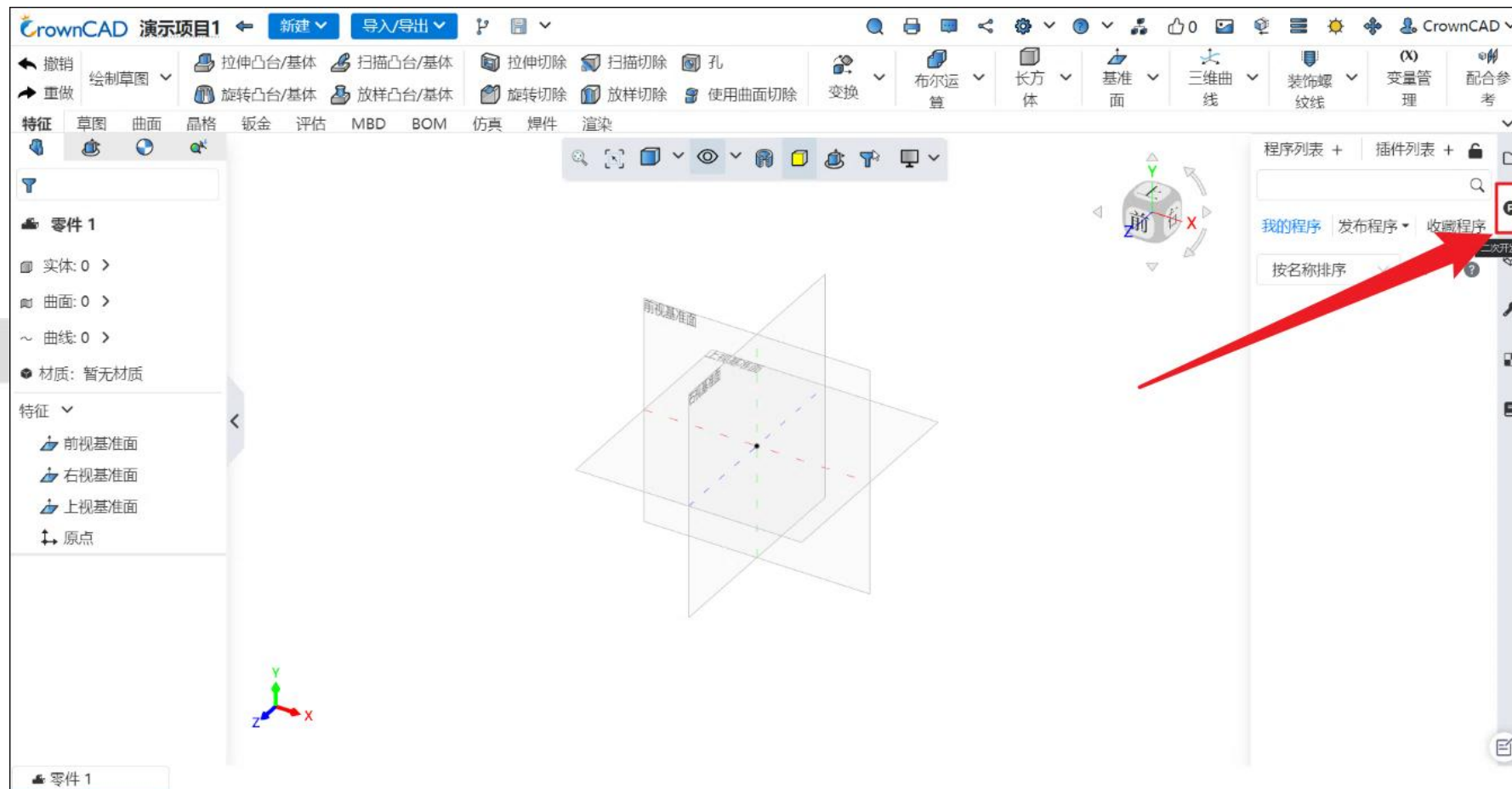
1.1 简介

- CrownCAD 二次开发是在 CrownCAD 平台的基础上，进行功能定制和扩展的开发过程。
- 主要目的是提高软件系统的灵活性和可扩展性，同时降低开发和维护成本。
- 通过二次开发，企业可以快速响应业务需求，增加新功能和改进现有功能。



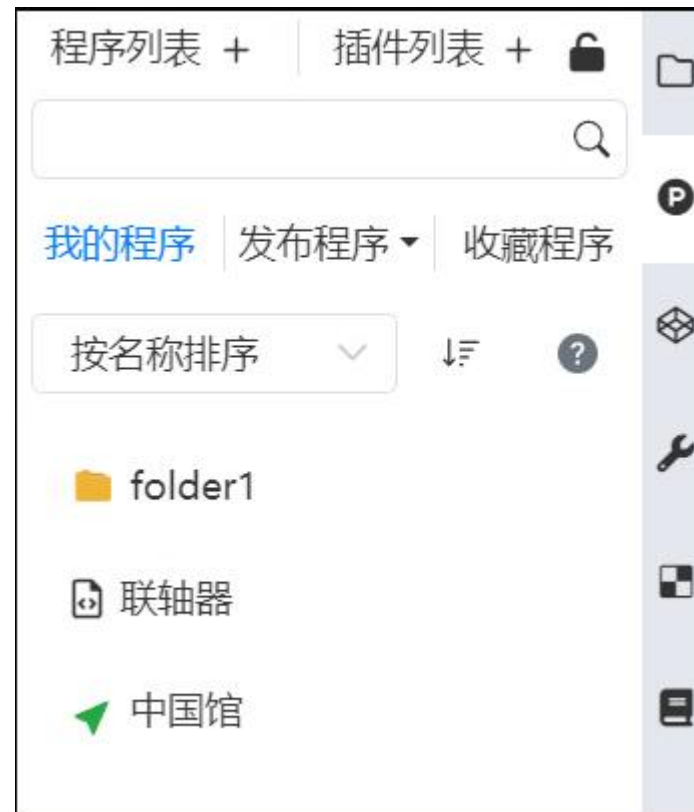
1.2 功能入口

➤ 工作空间右侧导航栏，点击  图标即可使用二次开发功能。



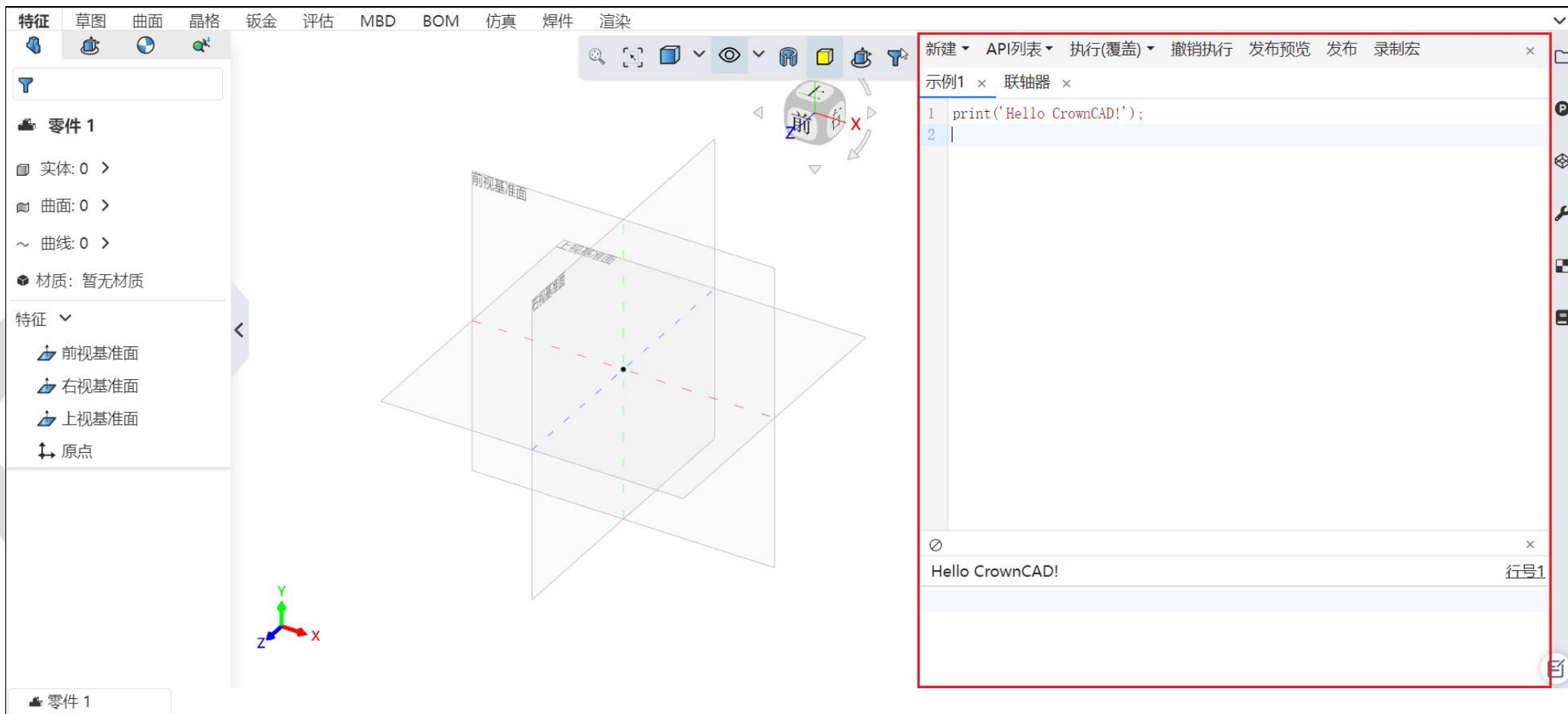
1.3 功能介绍

1. **新建程序**: 点击程序列表后的 + 按钮新建程序或文件夹;
2. **新建插件**: 点击插件列表后的 + 按钮新建插件或插件集;
3. **程序搜索**: 搜索我的程序或发布程序;
4. **我的程序**: 自己创建的程序列表;
5. **发布程序**: 自己发布的程序、其他用户公开发布的程序列表;
6. **收藏程序**: 自己收藏的程序列表;
7. **排序**: 程序列表排序方式;
8. **帮助手册**: ? 帮助手册入口;
9. **文件夹**: 📁 文件夹;
10. **程序**: 📄 二次开发程序;
11. **发布程序**: 🟢 已发布的程序。



1.4 编辑器面板

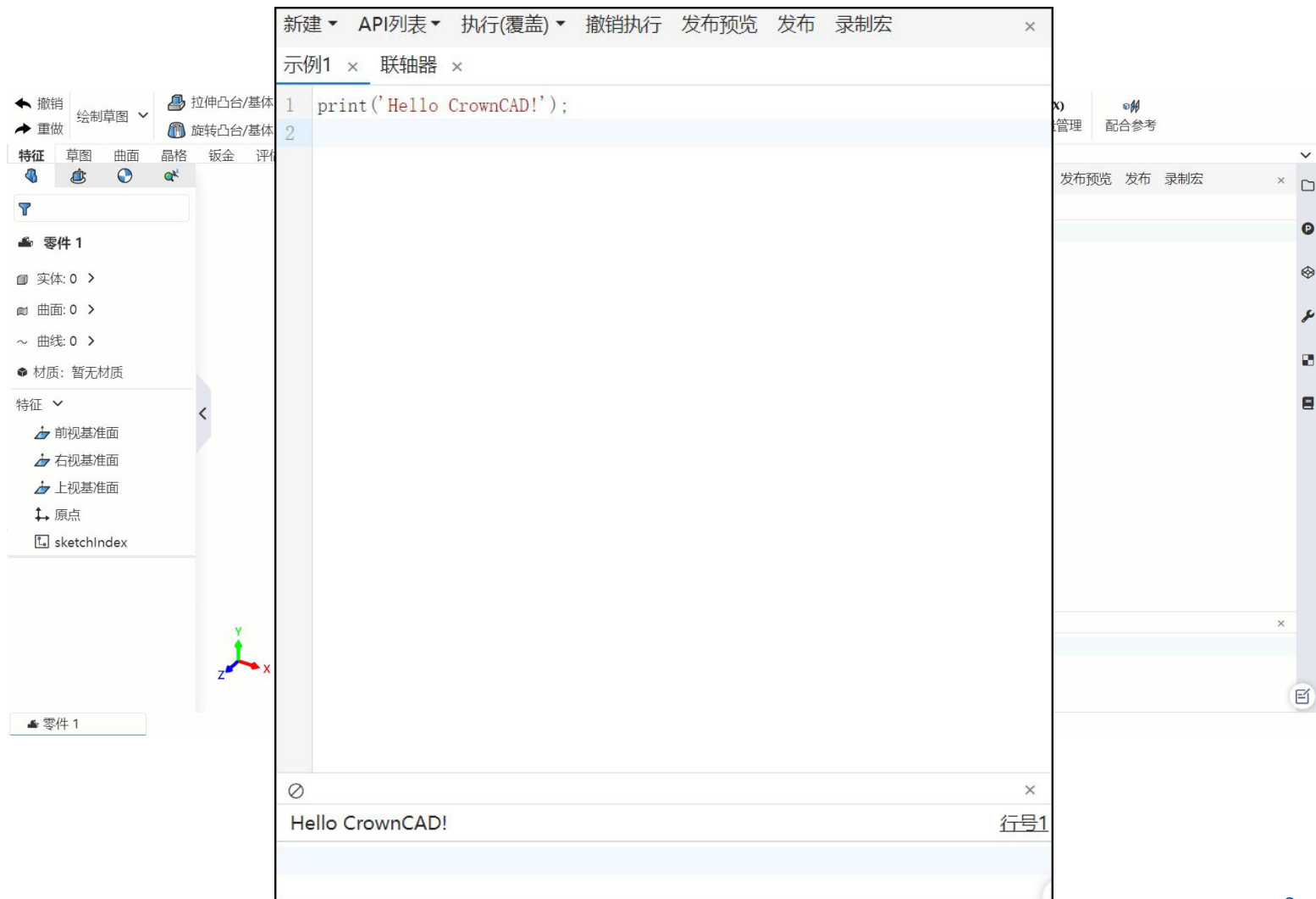
➤ 编辑器面板主要由工具栏、标签栏、脚本编辑器和控制台四部分组成。



1.4 编辑器面板

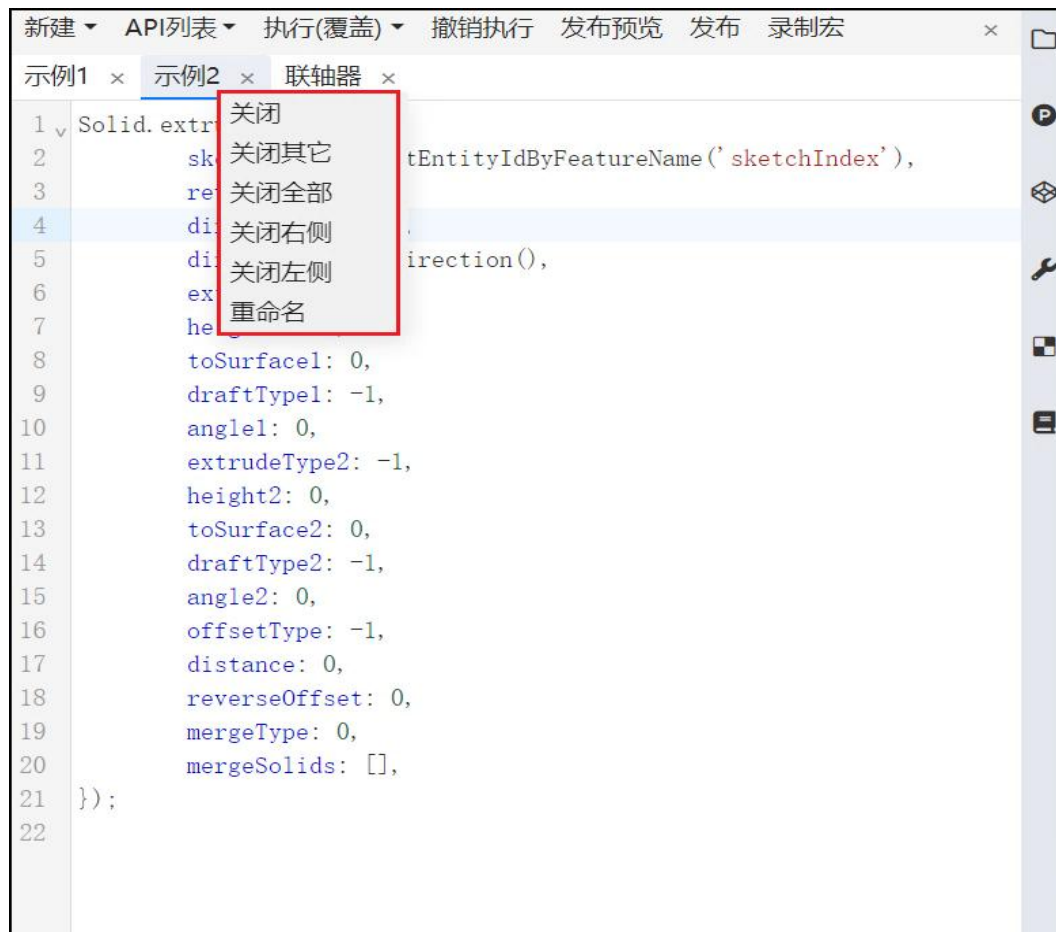
➤ 工具栏：脚本编辑的主要功能菜单

- 新建程序、上传图片；
- API 列表检索、插入 API 模板；
- 覆盖、追加执行程序；
- 撤销程序；
- 预览、发布程序；
- 脚本录制。



1.4 编辑器面板

➤ **标签栏：**程序标签页，支持打开多个程序。



1.4 编辑器面板

➤ 脚本编辑器：编写二次开发脚本的区域

- 支持语法高亮;
- 脚本格式化;
- 代码折叠;
- 脚本自动填充;
- 悬停提示;
- 右键菜单;
- 等等。



1.4 编辑器面板

- **控制台：**显示程序的输出信息、报错信息
 - 追加输出；
 - 清空控制台；
 - 执行程序段(Ctrl+Enter执行)。



目录

CONTENTS

02 语法规则



2.1 数据类型

- 数据类型可分为四大类：
 - **基本数据类型**：Integer(整数)、Number(浮点型)、Boolean(布尔类型)、String(字符串)、List(数组)、KVObject(字典);
 - **特殊数据类型**：Point(点)、Direction(方向)、Axis(轴)、Variable(变量);
 - **Enum 数据类型**：EntityType(实体类型)、ElementType(元素类型)、DocumentType(文档类型)、VariableType(变量类型)、VariableUnit(变量单位)、QueryType(拾取元素类型);
 - **几何数据类型**：Vector3(三维向量)、Matrix4(四阶矩阵)。

2.2 基本数据类型

类型	标识	示例	复制方式
整数	Integer	123	值复制
浮点型	Number	123.456 、 123	值复制
布尔类型	Boolean	true、 false	值复制
字符串	String	"string"、 'string'	值复制
数组	List	[]、 [1,2,3]	引用复制
字典类型	KVObject	{ }、 {temp1: 123, "temp2":456}	引用复制

2.3 特殊数据类型

类型	标识	定义方式	复制方式
点	Point	<code>new Point(x,y,z); //三维点</code> <code>new Point(x,y); //二维点</code>	值复制
方向	Direction	<code>new Direction(lineId); //通过直线 Id 创建</code> <code>new Direction(pnt1, pnt2); //通过两点创建</code>	值复制
轴	Axis	<code>new Axis(lineId); //通过直线 Id 创建</code> <code>new Axis(pnt1, pnt2); //通过两点创建</code>	值复制
变量	Variable	<code>new Variable(100); // 通过常量创建</code> <code>new Variable('varRef'); //通过变量名创建</code>	值复制

2.4 Enum 数据类型

➤ EntityType(实体类型)

元素	Solid	Sketch	Surface	Curve	DatumPlane	DatumLine
含义	三维体	草图	三维曲面	三维曲线	基准面	基准线

➤ ElementType(元素类型)

元素	Vertex	Edge	Face	Point	Dimension	Curve	Surface
含义	顶点	边	面	点	约束	曲线	曲面

➤ DocumentType(文档类型)

元素	Part	Assembly	Drawing	Mesh
含义	零件	装配	工程图	网格

2.4 Enum 数据类型

➤ QueryType(拾取元素类型)

元素	Instance	Feature	Solid	Shell	Sketch	DatumPlane
含义	装配实例	特征	实体	三维曲面	草图	基准面
元素	DatumLine	Surface	Face	Curve	Edge	Vertex
含义	基准线	平面	面	曲线	边线	顶点
元素	OriginPoint	PointOnCurve	Point			
含义	坐标源点	线上一点	草图点			

2.5 几何数据类型

类型	标识	定义方式	属性	复制方式
三维向量	Vector3	<pre>// 空参构建 [0, 0, 0] var b = new Vector3(); // 参数构建[0, 1, 0] var a = new Vector3(0, 1, 0); // 通过在程序中定义的点构建[p.x, p.y, p.z] var pnt = new Point(); var vec = new Vector3(pnt);</pre>	<pre>.x // 三维向量的 x 值, Number 类型 .y // 三维向量的 y 值, Number 类型 .z // 三维向量的 z 值, Number 类型</pre>	引用复制
四阶矩阵	Matrix4	<pre>// 空参构造一个 4 x 4 的单位矩阵 var mat = new Matrix4();</pre>	<pre>.elements // 使用一维数组 模拟存储四阶矩阵</pre>	引用复制

2.6 语法格式 - 变量赋值、if 语句、方法定义

变量赋值	if语句	方法定义
<pre>var a = 10; var b = 'hello' + a; var c; c = true; var list = []; list.add(a); var object1 = {'k1': 1, 'k2': 2}; object1['k1'] = 3; object1.put('k3',3);</pre>	<pre>if (a > 10) { b = a + 1; } else if (a > 5) { b = a-2; } else { b = a; }</pre>	<pre>function foo (arg1) { var a = arg1 + 5; return a; }</pre>

2.7 语法格式 - 循环语句

for循环	while循环
<pre>//for 循环 for (var i = 0; i < 10; ++i) { print(i) }</pre> <pre>//foreach 循环 var list = [1,2,3]; foreach (var item : list) { print(item); }</pre>	<pre>//while 循环 while (i < 100) { if (a == 10) { i++; continue; } if (a > 20) { break; } i++; }</pre>

2.8 语法格式 - 程序导入、导出

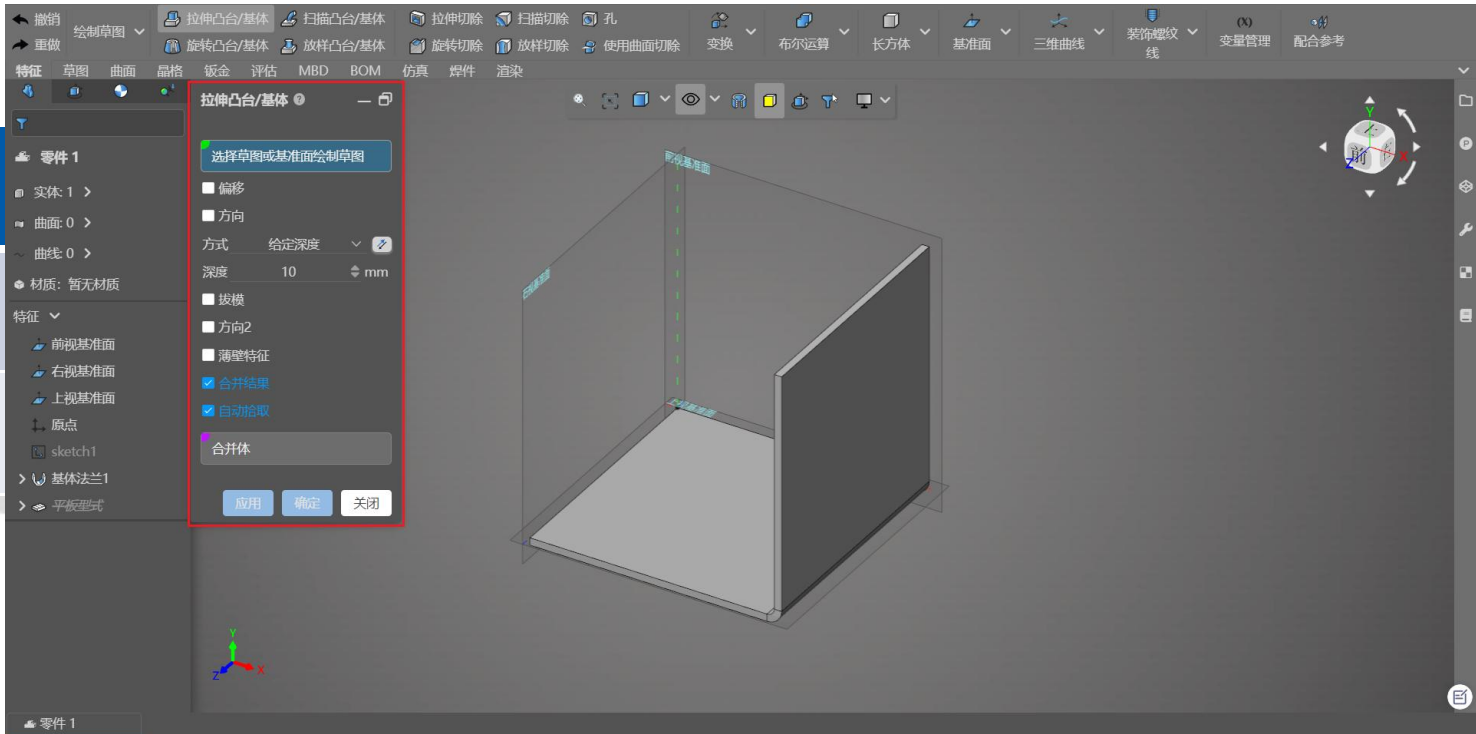
程序导出	程序导入
<pre>// 导出 export var b = 100.0; export function function2() { print("function2()"); } var a = 200; function function1(){ print("function1()"); } export {a, function1};</pre>	<pre>// 导入 // import * from "./导出"; // 导入目标程序中的导出的全部变量和方法 import { function1, function2 as function3, a as aa, b } from "./导出"; function1(); function3(); print(aa); print(b);</pre>

2.9 UI 布局组件 - 窗口组件

➤ 窗口组件是声明 UI 定义的开始，所有 UI 的定义都要定义在 UI 窗口内部，同时 UI 要定义在程序的最上面。

➤ @ui(height = "1000px", width="500px") {

.....

参数名		是否必须
width		否
height		否

2.10 UI布局组件 - 块状布局组件

```
@block_row(width = "1000px", height = "500px") { // 声明行元素
    @block_column(width = "200px") { // 声明列元素
        .....
    }
}
```

参数名	参数含义	参数类型	默认值	是否必须
title	块元素的名字	字符串	空	否
wigth	块的宽度（可填写：分辨率:'500px'，百分比：'100%'）	字符串	父元素宽度	否
height	块的高度（可填写：分辨率:'500px'，百分比：'100%'）	字符串	父元素高度	否

2.11 UI 布局组件 - 选项卡组件

```
// 声明当前开始定义 Tab 标签，内部使用不同的 tab 定义不同的标签
@tab {
    tab("页面 1") {} // 在 tab()内定义的是当前标签页的名字 - 必须
    tab("页面 2") {} // 声明不同标签页面
    var use = "页面 1"; // 声明默认打开的标签页面的名字 - 必须
}
```



2.12 UI 布局组件 - 折叠组件

- 某些情况下隐藏显示部分的 UI 窗口内容，可以选择使用该组件，定义在该组件内部的 UI 内容，用户可以选择展示与折叠隐藏。
- `@block("折叠组件", expanded = true) { }`

参数名	参数含义	参数类型	默认值	是否必须
(缺省)	折叠页的名字	字符串	'折叠组件'	是
expanded	是否默认展开	布尔类型	true	否

2.12 UI 布局组件 - 折叠组件

- 某些情况下隐藏显示部分的 UI 窗口内容，可以选择使用该组件，定义在该组件内部的 UI 内容，用户可以选择展示与折叠隐藏。
- `@block("折叠组件", expanded = true) { }`



页面1	页面2
输入框0	10.5
整数0	10
选择...	选项1
字符串0	str
折叠页面 >	
☒ 覆盖当前文档	
<button>确定</button> <button>关闭</button>	

2.13 基础 UI 组件 - 输入框

```
@input('数字', bind = 'number', default = 10.0, min = -50.0, max = 50.0);  
@input_integer('整数', bind = 'integer', default = 10, min = 0, max = 100);  
@input_string('字符串', bind = 'string', default = 'string');
```

参数名	参数含义	参数类型	默认值	是否必须
(缺省)	输入框的显示名	字符串		是
bind	定义参数在程序中使用的变量名	字符串		是
default	输入框默认值			是
max	输入框限定的最大值	数字	无穷大	否
min	输入框限定的最小值	数字	无穷小	否

2.13 基础 UI 组件 - 输入框

```
@input('数字', bind = 'number', default = 10.0, min = -50.0, max = 50.0);  
@input_integer('整数', bind = 'integer', default = 10, min = 0, max = 100);  
@input_string('字符串', bind = 'string', default = 'string');
```



test_ui	
输入框	10.5
输入框	10.5
整数	10
字符串	str
选择框	选项1
☒ 单选框	

2.14 基础 UI 组件 - 单选框

```
@checkbox('单选框', bind = "check", default = true);
```

参数名	参数含义	参数类型	默认值	是否必须
(缺省)	输入框的显示名	字符串	'单选框'	是
bind	定义参数在程序中使用的变量名	字符串	'check'	是
default	单选框默认值	布尔类型	true	是

2.14 基础 UI 组件 - 单选框

```
@checkbox('单选框', bind = "check", default = true);
```

test_ui

输入框	10.5	↑↓
输入框	10.5	
整数	10	↑↓
字符串	str	
选择框	选项1	▼
☒ 单选框		

2.15 基础 UI 组件 - 下拉选择框

```
var select = { "选项 1": 10, "选项 2": 'string', '选项 3': false, '选项 4': 10.11};  
@select('下拉框', bind = "option", default = select['选项 1']);
```

参数名	参数含义	参数类型	默认值	是否必须
(缺省)	输入框的显示名	字符串	'下拉框'	是
bind	定义参数在程序中使用的变量名	字符串	'select'	是
default	下拉框默认值	对象的取值		是

2.15 基础 UI 组件 - 下拉选择框

```
var select = { "选项 1": 10, "选项 2": 'string', '选项 3': false, '选项 4': 10.11};  
@select('下拉框', bind = "option", default = select['选项 1']);
```



2.16 基础 UI 组件 - 图片组件

- 当需要图片显示在 UI 面板上时，可以通过图片组件进行显示，图片需要提前上传，使用时引用图片的 id 来进行显示。
- `@image(id='0', width='100%', height='100%');`

参数名	参数含义	参数类型	默认值	是否必须
id	图片上传后返回的 Id	字符串	'0'	是
width	图片宽度 (可填写分辨率: '1000px', 百分比: '100%')	字符串	父元素宽度	否
height	图片高度 (可填写分辨率: '1000px', 百分比: '100%')	字符串	父元素高度	否

2.17 基础 UI 组件 - 文字组件

- 可以通过文字组件来实现对于文本内容的描述信息。
- @span(value = '文字内容');

参数名	参数含义	参数类型	默认值	是否必须
value	显示的文字内容	字符串	'文字内容'	是

2.18 基础 UI 组件 - 表格组件 - @table

@table(name="定义表格", width="1000px", height="500px") {}

参数名	参数含义	参数类型	默认值	是否必须
name	表格的名字	字符串	'表格'	否
width	表格整体宽度（可填写：分别率： '500px', 百分比：'100%'）	字符串	撑宽	否
height	表格整体高度（可填写：分别率： '500px', 百分比：'100%'）	字符串	撑高	否

2.18 基础 UI 组件 - 表格组件 - @row

@row(align = "center", type = "text", head = true) {}

参数名	参数含义	参数类型	默认值	是否必须
align	行内默认对齐方式 (left: 左对齐, center: 居中, right: 右对齐)	字符串	'center'	否
type	行内默认类型 (text: 文本类型, number: 数字类型, integer: 整数类型, string: 字符串类型, boolean: 单选框, select: 下拉框)	字符串	'text'	否
head	是否为表头栏	字符串	false	否

2.18 基础 UI 组件 - 表格组件 - @cell

@cell (value = 100, bind = "cell", type="number");

参数名	参数含义	参数类型	默认值	是否必须
value	单元格的值	与 type 的类型 相对应		是
bind	定义参数在程序中使用的变量名	字符串	'cell'	type!="text" 时为必须参 数
type	当前单元格的类型	字符串	'text' 或上一层 定义	否
align	当前单元格对齐方式	字符串	'center' 或上 一层定义	否

2.18 基础 UI 组件 - 表格组件

// 声明一个表格

```
@table(name="定义表格", width="1000px", height="500px") {  
  //声明表格一行和默认属性  
  @row(align = "center", type = "text", head = true){  
    @cell (value = "基本尺寸"); // 定义文本内容表格  
    @cell (value = 100, bind = "cell", type="number");  
  }  
}
```

2.18 基础 UI 组件 - 表格组件

test_table

基本尺寸	mm	基本尺寸	mm	基本尺寸	mm
基本尺寸	mm	基本尺寸	mm	基本尺寸	mm
D	300	D1	220	D2	300
D3	300	H	220	H1	300
H2	300	L	220	L1	300
L2	300	L3	220	L4	300
L5	235	W	150	W1	450
W3	150	W4	500	f1	7

☒ 覆盖当前文档

2.19 基础 UI 组件 - 交互框组件

- 在 UI 面板中定义交互框组件，可以实现对模型中的元素的拾取，并在程序中可以使用拾取的元素内容。

注: 交互框组件的使用只可以在发布预览和发布之后使用，即需要通过面板执行，不允许直接执行

- `@query(' 顶 点 ', bind='query', filters=[QueryType.Vertex], minCount=2, maxCount=2, tipMsg="请选择两个顶点");`

参数名	参数含义	参数类型	默认值	是否必须
(缺省)	交互框的名字	字符串	'交互框'	是
bind	参数在程序中使用的变量名	字符串	'query'	是
filters	交互框允许拾取的元素类型	数组	[]	是
minCount	允许拾取的最少元素数量	整数	1	否
maxCount	允许拾取的最多元素数量	整数	100	否
tipMsg	交互框的提示信息	字符串	'请选择相关元素'	否

目录

CONTENTS

03 API 使用



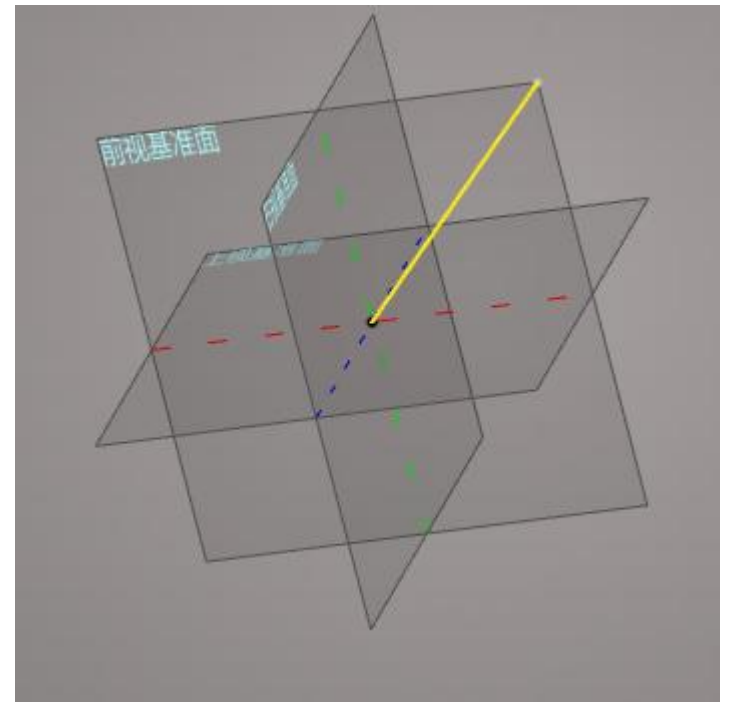
3.1 API 概述

- 二次开发的建模 API 在语言形式上采用函数调用 (MethodCall) 的形式，例如拉伸命令为 Solid.extrude()，前部分是模块名，后部分是方法名；
- 创建特征的接口有返回值，返回特征的信息：{'instanceId': '', 'entityId': 0, 'id': 0, 'name': ''}
 - instanceId：装配内插入的实例 Id；
 - entityId：零件内实体、草图的 Id；
 - elementId：实体内点、线、面等元素的 Id；
- 目前已有基准、草图、实体、曲线、曲面、钣金、装配、查询、公共和数学模块。

3.2 草图模块 - 创建草图

```
var frontDatumId = Query.getEntityIdByFeatureName('前视基准面');  
//创建草图  
Sketch.createSketch('自定义草图', frontDatumId);  
var startPnt = new Point(0, 0, 0);  
var endPnt = new Point(100, 100, 0);  
var lineId = Sketch.createLine(startPnt, endPnt);  
Sketch.exitSketch();
```

- 创建草图后记得要**退出草图**，否则程序会执行会失败；
- 绘制草图时需要注意定义的 Point 是否在所选的基准面上，如果不在基准面上，则创建不出草图；
- 草图内创建的元素为单一元素时，接口有返回值，返回值为元素 id
 - 创建线、圆、椭圆、圆弧等有返回值；
 - 创建矩形接口，内核返回的是多个线和点，这一类接口没有返回值。



3.3 基准模块 - 创建基准线

```
var frontDatumId = Query.getEntityIdByFeatureName('前视基准面');
```

```
//创建草图线
```

```
Sketch.createSketch('自定义草图', frontDatumId);
```

```
var startPnt = new Point(0, 0, 0);
```

```
var endPnt = new Point(100, 100, 0);
```

```
var lineId = Sketch.createLine(startPnt, endPnt);
```

```
Sketch.exitSketch();
```

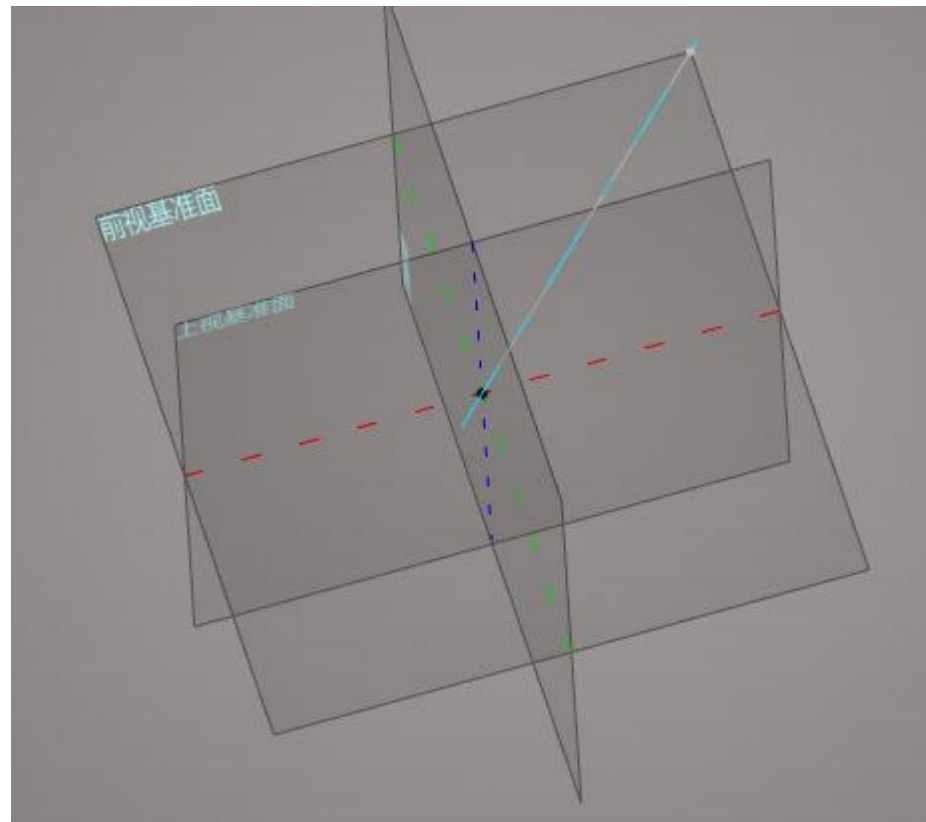
```
//创建基准线
```

```
var datumLine = Datum.createLine("", {
```

```
  referenceType: 3,
```

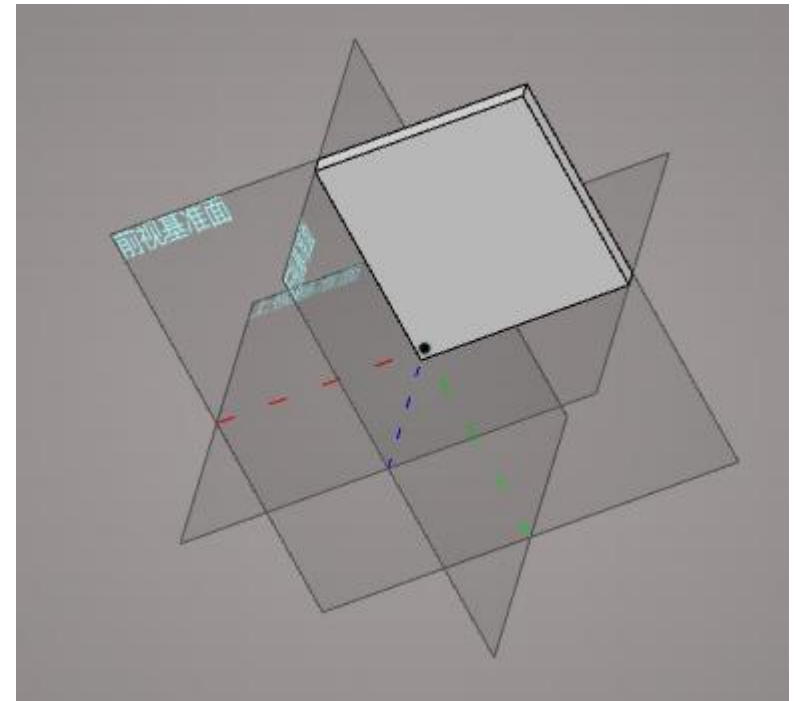
```
  referenceEntities: [lineId],
```

```
});
```



3.4 实体模块 - 拉伸凸台

```
var frontDatumId = Query.getEntityIdByFeatureName('前视基准面');  
//创建草图  
Sketch.createSketch('自定义草图', frontDatumId);  
var startPnt = new Point(0, 0, 0);  
var endPnt = new Point(100, 100, 0);  
Sketch.createRectangle(startPnt, endPnt);  
Sketch.exitSketch();  
var sketchId = Query.getEntityIdByFeatureName('自定义草图');  
//拉伸凸台  
var extrudeInfo = Solid.extrude('extrude',{  
    sketch: sketchId,  
    reverse: 0,  
    height1: 10,  
});
```



3.5 曲线模块 - 三维曲线

//创建曲线

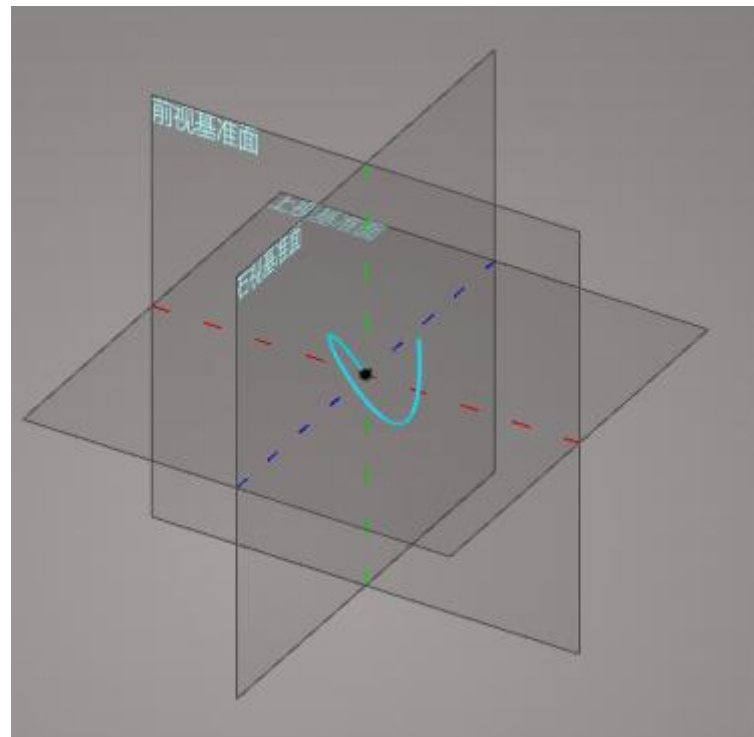
```
var pnt1 = new Point(25,25,0);
```

```
var pnt2 = new Point(25,0,25);
```

```
var pnt3 = new Point(0,25,25);
```

```
var basePnt = new Point(0,0,0);
```

```
var curveInfo = Curve.createCurveByInterpolationPoints('三维曲线',{  
  pickPnts: [pnt1, pnt2, pnt3, basePnt],  
  isClosed: 0,  
});
```



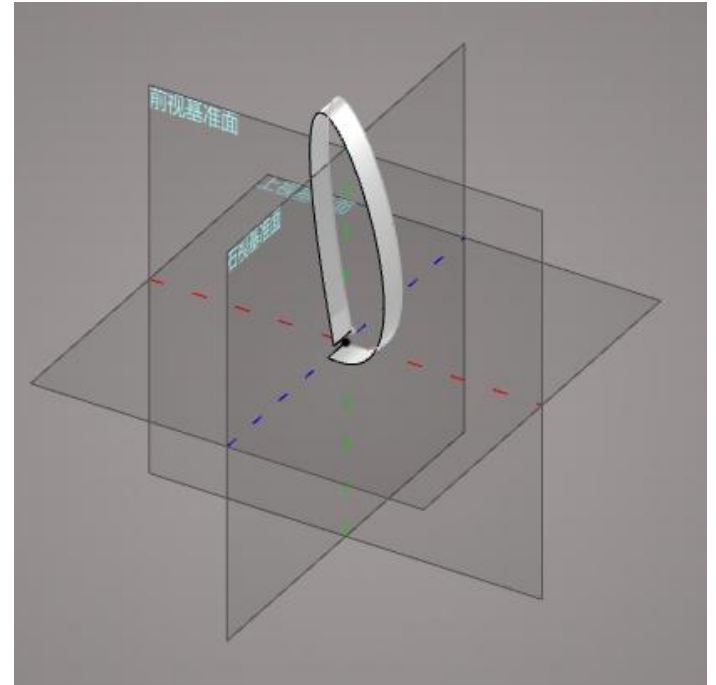
3.6 曲面模块 - 拉伸曲面

//曲线

```
Sketch.createSketch('草图样条曲线', frontDatumId);  
var p1 = new Point(0,0);  
var p2 = new Point(10,0);  
var p3 = new Point(10,110);  
var p4 = new Point(3,7);  
var pickPnt = [p1, p2, p3, p4];  
var curveld = Sketch.createIntpCurve(pickPnt);  
Sketch.exitSketch();  
var sketchId = Query.getEntityIdByFeatureName('草图样条曲线');
```

//拉伸曲面

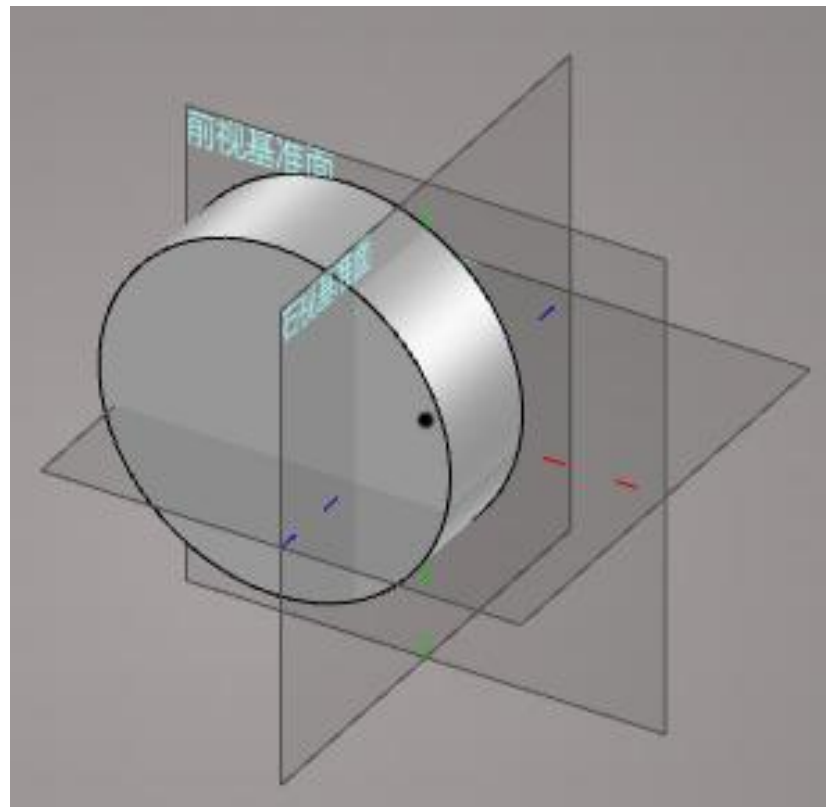
```
Surface.extrudeSurface('拉伸曲面',{  
  curvelds: [curveld],  
  reverse: 0,  
  height1: 15,  
});
```



3.7 装配模块 - 插入实例

// 插入实例

```
var res = Assembly.insertComponent('insertComponent',{  
  docName: 'Part 3',  
  position: new Point(0,0,0),  
  boxCenter: 0,  
  projectId: '63156019f09236609d9c4f96',  
});
```



目录

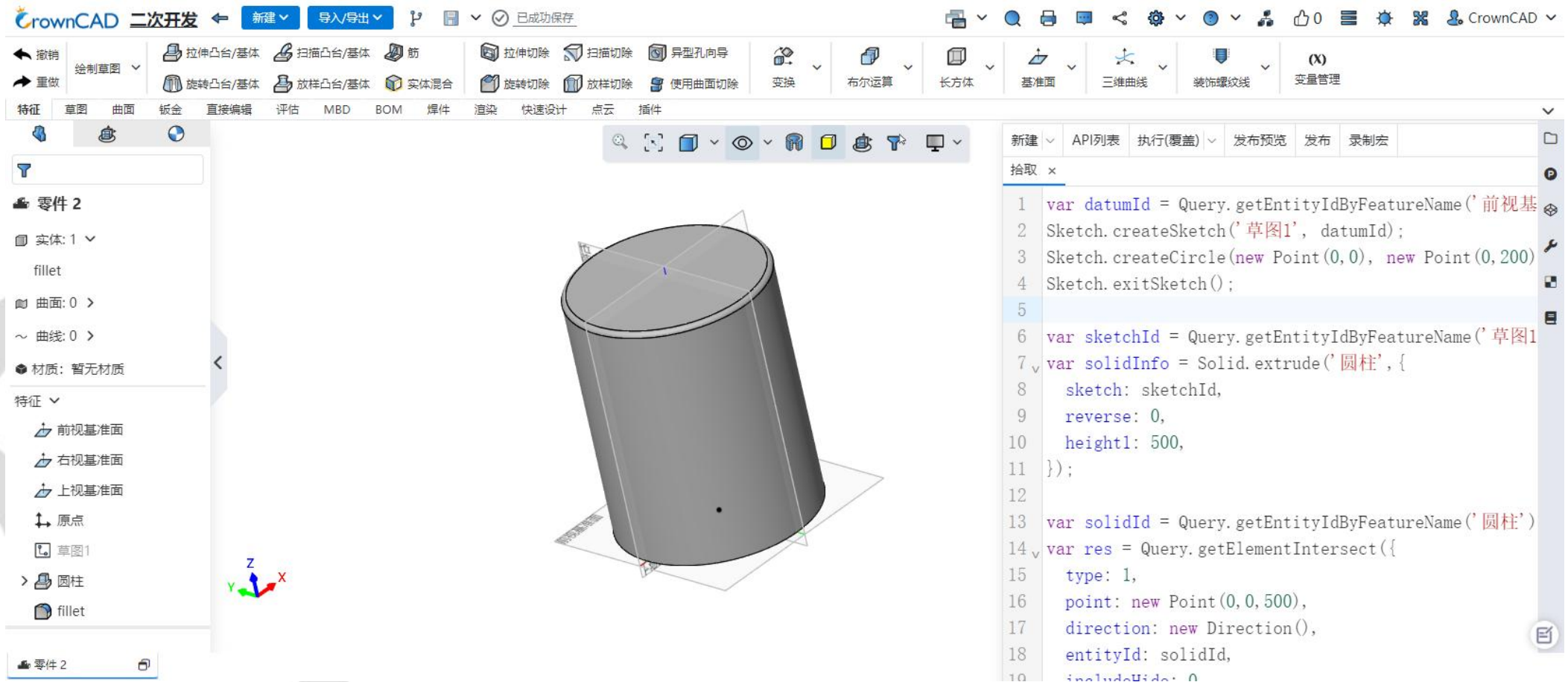
CONTENTS

04 示例程序



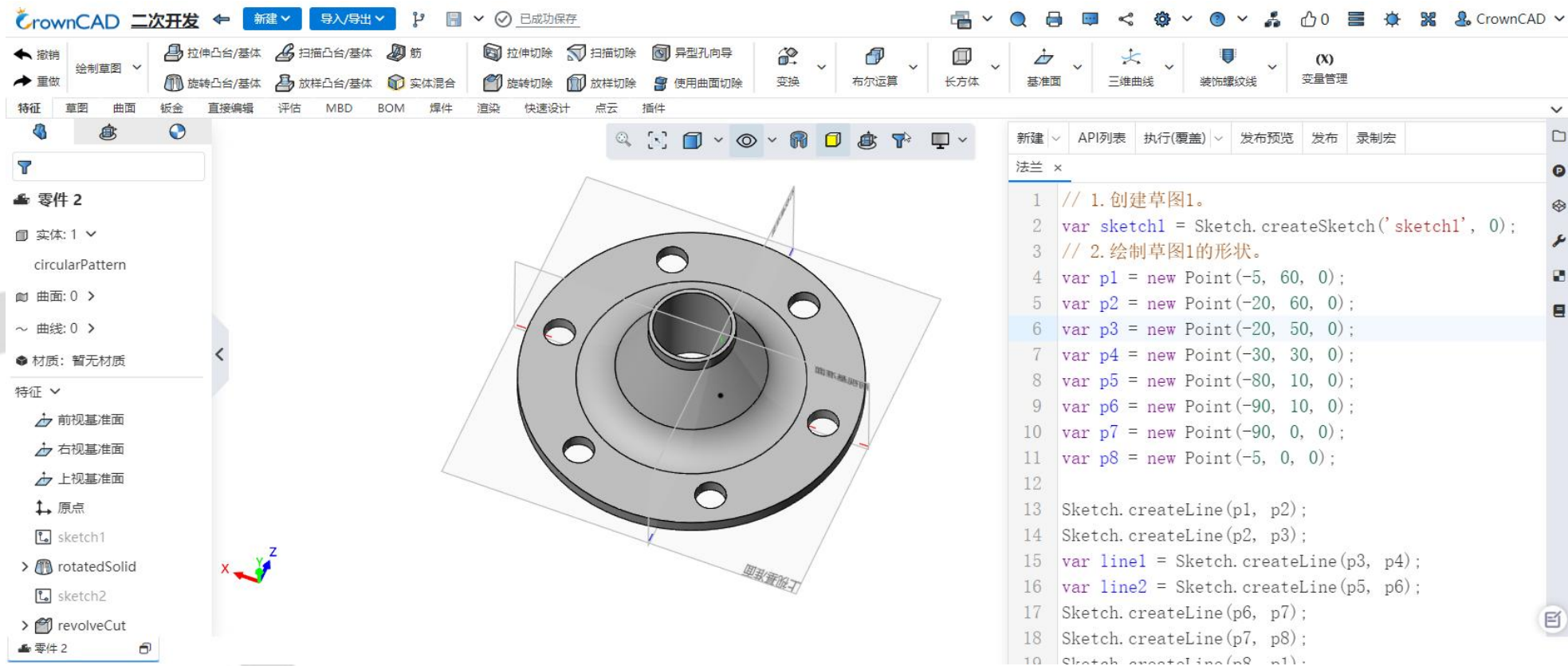
4.1 圆柱

➤ 通过程序绘制一个圆柱，并添加圆角。



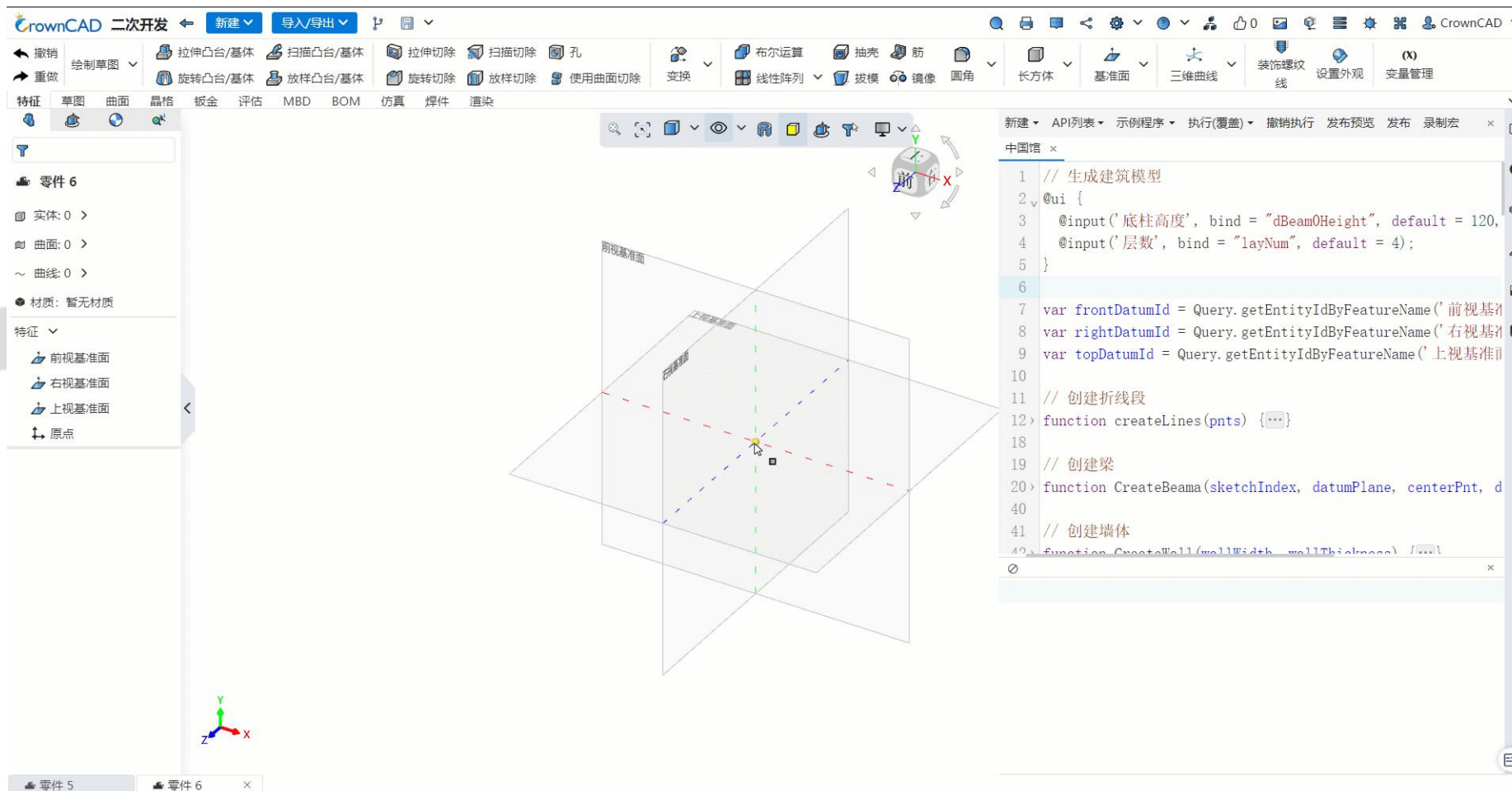
4.2 法兰绘制

➤ 通过程序绘制法兰。



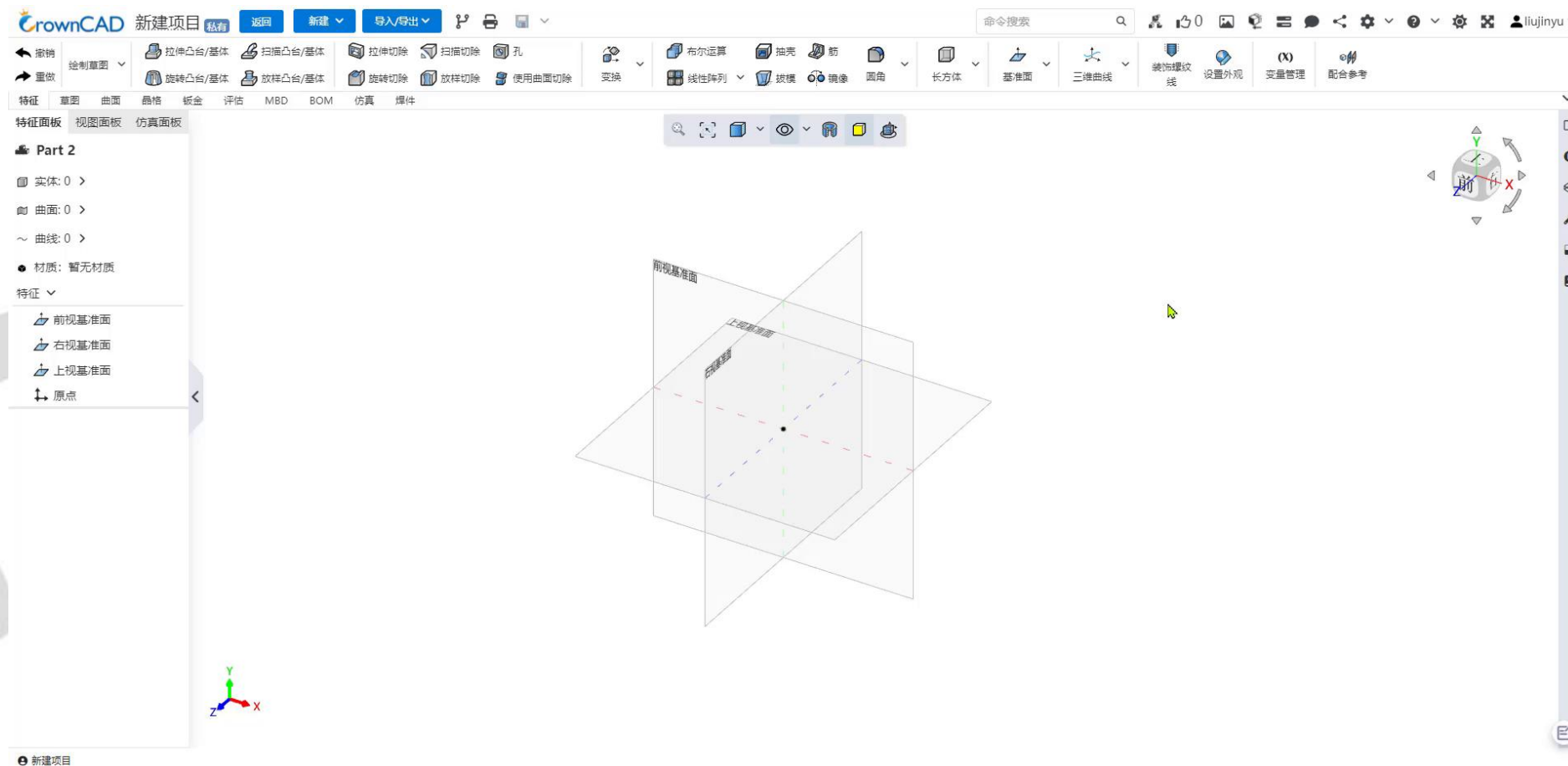
4.3 完整程序示例

➤ 通过输入底柱高度和层数，驱动出中国馆模型。



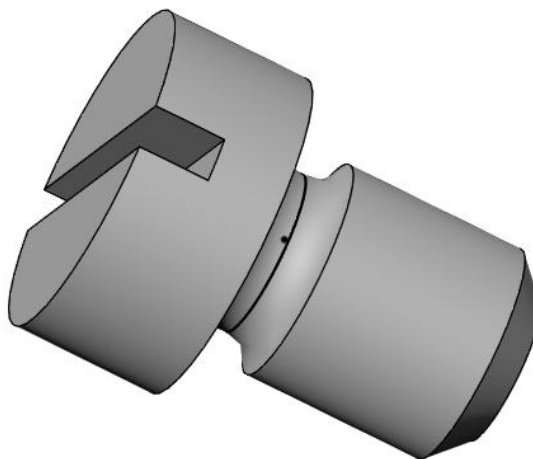
4.4 插件模式二次开发

➤ 插件开发可提供更高的灵活度和能力范畴，支持做更复杂的应用功能。



4.5 如何实现螺塞的绘制？

- 通过输入对应的尺寸参数，驱动生成不同规格的螺塞。



目录

CONTENTS

05 问题答疑





华云三维微信公众号

www.crowncad.com

